Mira 评审说明书

proof-first judging brief | 面向评委的项目说明，而非品牌宣言

一句话定位
Mira 是一个基于 OpenClaw 的主动感受型 AI 伴侣，使用眼镜、可穿戴信号与家居执行面，完成**低打扰、可解释、可控**的关怀闭环。

Perception	Memory	Judgment	Action
眼镜画面、心率、环境上下文	短期状态与近期事件记忆	判断是否值得打扰	镜片提示、耳机/手表、家居执行

评委先看这三点

- 本项目主打的不是“更复杂的 AI”，而是**更实际的 care loop**。
- 当前材料把“已实现 / 部分实现 / 愿景”明确分层，避免把概念包装成现成能力。
- 本次评审主打闭环是：**公开表达/演讲前支持与 wearable -> Home Assistant** 两条已验证路径。

一、主问题与主用户

把叙事收束到一个评委最容易判断实际价值的高频时刻

主场景锚点
公开表达前的高压时刻。用户在上台、路演、演讲前往往不会主动对 AI 说“我现在很紧张”，但这个时刻恰恰最适合主动感知、最容易观察、也最容易形成低打扰闭环。

当前 AI 的问题

- 大多依赖用户显式开口。
- 即使会主动发消息，也常常不知道用户当下状态。
- 在真正紧张或疲惫时，用户往往不想先发起对话。

Mira 的区别

- 先感知，再决定是否值得行动。
- 默认沉默；不确定时不打扰。
- 动作不是聊天优先，而是**最小、最合适的支持**。

Current AI needs explicit ask

Mira acts before the ask, but only when appropriate

为什么先锚定这一类用户？

- 这是一个**单一、高频、容易展示**的痛点，适合 hackathon 评审。
- 既能体现主动感知，也能体现“低打扰”和“分寸感”。
- 其他场景如独居老人、回家关怀、儿童安全可作为**未来扩展场景**，不与 MVP 抢叙事中心。

未来扩展方向（本次不作为主判分对象）
独居老人趋势守护、下班回家照护、儿童危险姿态监测、社交记忆与长期关系理解。

二、当前实现分层

把 marketing claim 变成 engineering status

已实现 / 已验证

- Rokid 观察事件接入与 ‘POST /v1/observe‘
- 心率事件到家庭动作的 demo 路径
- 待确认动作缓存与确认接口
- Memory ledger / transient memory 基础链路
- Home Assistant 执行插件与 demo 场景
- outbound policy 文件化与测试

部分实现 / 可演示

- 演讲前鼓励的现场 narrative 闭环
- 多模态判断与记忆检索拼接
- 低风险主动提醒 / 私聊型提醒
- 眼镜侧陪伴式输出体验

愿景 / 扩展方向

- 独居老人趋势型守护
- 回家时刻自动照护编排
- 儿童危险姿态长期监护
- 更稳定的社交记忆与长期陪伴

本次主打闭环

- **闭环 A**：演讲前主动感知 -> 最小鼓励输出
- **闭环 B**： wearable / 观察事件 -> OpenClaw -> Home Assistant 执行

为何这样分层?
评委需要看到的是“这次比赛里到底做成了什么”。因此本稿明确区分：**Implemented / Partially implemented / Vision**，避免把所有场景写成一个语气层级。

三、主 demo：演讲鼓励场景全链路

从“有感染力的故事”改成“可解释的闭环”

场景目标
用户上台前不会主动说“我很紧张”，但系统可以从视觉、穿戴状态和近期上下文中判断：此时是否值得给出一句**最小支持**。

1. Rokid 观察到用户已处于路演/演讲场景，周围存在 PPT 与观众等公开表达线索。
2. 可穿戴侧出现紧张信号，例如心率提升或身体状态变化。
3. 系统检索近期上下文与准备痕迹，形成“此刻确实值得支持”的判断。
4. Care policy 决定是否行动。若不确定，则保持沉默。
5. 若高置信度成立，则输出一句**最小、不过度暴露隐私**的鼓励信息。
6. 输出优先在眼镜 / 耳机等私密界面完成，避免公共暴露与过度打扰。

示例输出
“放轻松，你肯定可以做到的。深呼吸。你这两天做了很多准备，去拿下这个舞台。”

这页要证明什么？
- Mira 不是普通 chatbot。
- 它的价值不在“能不能聊”，而在“能不能在合适时机少说一句但更有用的话”。
- 它的关键能力是**先判断要不要行动**，而不是默认一直说话。

fallback 与边界
- 观察不充分时，不输出支持话术。
- 动作优先级：私密支持优于公开提醒。
- 中高风险外发动作保留确认步骤，不在该 demo 中自动扩张。

四、工程结构与可复现路径

从 philosophy 改成 data flow + component evidence

最简 pipeline

`frame+wearablesignal+context->observe->memory->policy->confirm/execute`

组件	作用	仓库证据
Rokid bridge gateway	接收观察事件、规范化响应、保存待确认动作	`services/rokid-bridge-gateway/src/routes/observe.ts` `.../routes/confirm.ts`
Memory layer	记录观察、确认与记忆检索上下文	`.../memory/sqliteMemoryLedger.ts` `.../store/transientMemory.ts`
Policy layer	对外发行为做 quiet hours、recipient scope 与风险规则控制	`services/rokid-bridge-gateway/config/outbound-policy.yaml` `.../outbound/outboundPolicyEvaluator.ts`
HA executor	把判断转成 Home Assistant / 多生态动作	`plugins/openclaw-plugin-ha-control/src/index.ts` `.../src/ecosystem.ts`
Demo scripts	打通可复现的 end-to-end 演示路径	`scripts/demo-sequence.sh` `scripts/demo-rokid-coffee-sequence.sh`

工程语言下的四层理解

- **Sensor ingestion:** 眼镜、可穿戴、脚本与设备事件进入系统。
- **Memory write / retrieve:** 观察事件写入 ledger，形成可追溯上下文。
- **Care policy engine:** 不是“看到就做”，而是先判断是否值得打扰。
- **Action router:** 动作最终路由到眼镜、通知或 Home Assistant 执行面。

五、价值证明页

不是“问题有多大”，而是“闭环有多真”

闭环	证据类型	实际价值
Wearable -> HA cooling	仓库 README 的 Demo 1 路径 `./scripts/demo-sequence.sh`	证明系统能把生理状态事件变成家庭执行响应，而不是只做聊天输出。
Rokid observe -> confirm -> HA	仓库 README 的 Demo 2 路径 `./scripts/demo-rokid-coffee-sequence.sh`	证明系统具备“观察 -> 待确认动作 -> 执行”的 inspectable action loop。
演讲鼓励 narrative	路演讲稿与硬件 demo deck	证明 Mira 的目标不是“更会说话”，而是“在高价值时刻少打扰、但更贴身”。

Judge takeaway
你什么都没说，Mira 仍然先注意到了你，并在不越界的前提下给出了帮助。

降低求助门槛
减少“用户必须先表达自己状态”的交互门槛。

把支持提前
把支持提前到关键时刻，而不是事后安慰。

系统可解释
动作路径可解释、可确认、可追溯，不是黑盒自动化。

六、安全与边界

把“我们重视安全”改成“当前有哪些明确规则”

当前 repo 中已文件化的规则

- 默认动作：**ask**
- 需要已知 recipient
- 默认要求 private channel
- quiet hours：23:00 – 08:00
- rate limits：每小时 3 次、每天 10 次
- caregiver escalation：默认 ask
- public / social posting：block
- secret / credential / raw private data：block

当前设计中强调的产品原则

- 默认沉默，不确定不行动
- 高风险外发需要确认
- 用户持有暂停 / 擦除 / 撤销权
- 家居执行应在 allowlist / whitelist 内进行
- 感知是为了照顾，不是为了监控

为何这一页重要?
Mira 的定义不是“更主动就更好”，而是在被允许的边界内、用可审计规则主动。这也是它与普通“全自动 agent”最关键的区别之一。

机制	证据
Outbound policy	`services/rokid-bridge-gateway/config/outbound-policy.yaml`
Confirmation path	`services/rokid-bridge-gateway/src/routes/confirm.ts`
Policy evaluation tests	`services/rokid-bridge-gateway/src/__tests__/outbound-policy.test.ts`

七、路线图与团队

从 hackathon MVP 到可发布 companion system

Hackathon now	聚焦 1–2 条主闭环：演讲鼓励、wearable -> HA、Rokid 观察 -> confirm -> action。
Next 1 month	把 care policy、日志、确认链路与设备执行面进一步产品化，让 demo 更 inspectable、更稳定。
3–6 months	扩展到回家关怀、独居守护、社交通知与 channel-agnostic communication，并把 Mira Core 做成更清晰的发布结构。

成员	本次聚焦职责
Rick	产品定义、路演叙事、用户体验、主场景 framing 与整体表达。
Junkai	后端链路、OpenClaw / Rokid / HA 接入、安全边界与执行链路。

评委最后 10 秒记住这三点

- Mira 证明的是“先感知、再判断、再行动”的 care loop，而不是聊天文案。
- 本次比赛已经打通两类主路径：演讲鼓励 narrative 与 wearable / observe -> confirm -> HA。
- 系统不是默认自动化，而是有 policy、确认链路与设备边界的可控执行。

最后一句

Mira 想证明的不是“AI 可以更炫”，而是：**AI 可以在合适的时候，更早一点懂你，更稳一点帮你。**

Relevant repo anchors: `README.md`, `OpenClaw/devbox/project/openclaw-ha-blueprint-memory/README.md`, `Readme/mira-pitch-script-v1.pdf`, `Readme/mira-track3-hardware-demo-deck.pdf`